

Chapter 12. Administering Kafka

Managing a Kafka cluster requires additional tooling to perform administrative changes to topics, configurations, and more. Kafka provides several command-line interface (CLI) utilities that are useful for making administrative changes to your clusters. The tools are implemented in Java classes, and a set of scripts are provided natively to call those classes properly. While these tools provide basic functions, you may find they are lacking for more complex operations or are unwieldy to use at larger scales. This chapter will describe only the basic tools that are available as part of the Apache Kafka open source project. More information about advanced tools that have been developed in the community, outside of the core project, can be found on the [Apache Kafka website](#).

AUTHORIZING ADMIN OPERATIONS

While Apache Kafka implements authentication and authorization to control topic operations, default configurations do not restrict the use of these tools. This means that these CLI tools can be used without any authentication required, which will allow operations such as topic changes to be executed with no security check or audit. Always ensure that access to this tooling on your deployments is restricted to administrators only to prevent unauthorized changes.

Topic Operations

The `kafka-topics.sh` tool provides easy access to most topic operations. It allows you to create, modify, delete, and list information about topics in the cluster. While some topic configurations are possible through this command, they have been deprecated, and it is recommended to use the more robust method of using the `kafka-config.sh` tool for configuration changes. To use the `kafka-topics.sh` command, you must provide the cluster connection string and port through the `--bootstrap-server` option. In the examples that follow, the cluster connect string is being run locally on one of the hosts in the Kafka cluster, and we will be using `localhost:9092`.

Throughout this chapter, all the tools will be located in the directory `/usr/local/kafka/bin/`. The example commands in this section will assume you are in this directory or have added the directory to your `$PATH`.

CHECK THE VERSION

Many of the command-line tools for Kafka have a dependency on the version of Kafka running to operate correctly. This includes some commands that may store data in ZooKeeper rather than connecting to the brokers themselves. For this reason, it is important to make sure the version of the tools that you are using matches the version of the brokers in the cluster. The safest approach is to run the tools on the Kafka brokers themselves, using the deployed version.

Creating a New Topic

When creating a new topic through the `--create` command, there are several required arguments to create a new topic in a cluster. These argu-

ments must be provided when using this command even though some of them may have broker-level defaults configured already. Additional arguments and configuration overrides are possible at this time, as well using the `--config` option, but are covered later in the chapter. Here is a list of the three required arguments:

`--topic`

The name of the topic that you wish to create.

`--replication-factor`

The number of replicas of the topic to maintain within the cluster.

`--partitions`

The number of partitions to create for the topic.

GOOD TOPIC NAMING PRACTICES

Topic names may contain alphanumeric characters, underscores, dashes, and periods; however, it is not recommended to use periods in topic names. Internal metrics inside of Kafka convert period characters to underscore characters (e.g., “topic.1” becomes “topic_1” in metrics calculations), which can result in conflicts in topic names.

Another recommendation is to avoid using a double underscore to start your topic name. By convention, topics internal to Kafka operations are created with a double underscore naming convention (like the `__consumer_offsets` topic, which tracks consumer group offset storage). As such it is not recommended to have topic names that begin with the double underscore naming convention to prevent confusion.

Creating a new topic is simple. Run `kafka-topics.sh` as follows:

```
# kafka-topics.sh --bootstrap-server <connection-string>:<port> --create --topic <string>
--replication-factor <integer> --partitions <integer>
#
```

The command will cause the cluster to create a topic with the specified name and number of partitions. For each partition, the cluster will select the specified number of replicas appropriately. This means that if the cluster is set up for rack-aware replica assignment, the replicas for each partition will be in separate racks. If rack-aware assignment is not desired, specify the `--disable-rack-aware` command-line argument.

For example, create a topic named “my-topic” with eight partitions that have two replicas each:

```
# kafka-topics.sh --bootstrap-server localhost:9092 --create
--topic my-topic --replication-factor 2 --partitions 8
Created topic "my-topic".
#
```

When using `kafka-topics.sh` in automation, you may want to use the `--if-not-exists` argument while creating new topics that will not return an error if the topic already exists.

While an `--if-exists` argument is provided for the `--alter` command, using it is not recommended. Using this argument will cause the command to not return an error if the topic being changed does not exist. This can mask problems where a topic does not exist that should have been created.

Listing All Topics in a Cluster

The `--list` command lists all topics in a cluster. The list is formatted with one topic per line, in no particular order, which is useful for generating a full list of topics.

Here's an example of the `--list` option listing all topics in the cluster:

```
# kafka-topics.sh --bootstrap-server localhost:9092 --list
__consumer_offsets
my-topic
other-topic
```

You'll notice the internal `__consumer_offsets` topic is listed here. Running the command with `--exclude-internal` will remove all topics from the list that begin with the double underscore mentioned earlier, which can be beneficial.

Describing Topic Details

It is also possible to get detailed information on one or more topics in the cluster. The output includes the partition count, topic configuration overrides, and a listing of each partition with its replica assignments. This can be limited to a single topic by providing a `--topic` argument to the command.

For example, describing our recently created “my-topic” in the cluster:

```
# kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic my-topic
Topic: my-topic PartitionCount: 8      ReplicationFactor: 2    Configs: segment.bytes=1
    Topic: my-topic Partition: 0    Leader: 1      Replicas: 1,0    Isr: 1,0
    Topic: my-topic Partition: 1    Leader: 0      Replicas: 0,1    Isr: 0,1
    Topic: my-topic Partition: 2    Leader: 1      Replicas: 1,0    Isr: 1,0
    Topic: my-topic Partition: 3    Leader: 0      Replicas: 0,1    Isr: 0,1
    Topic: my-topic Partition: 4    Leader: 1      Replicas: 1,0    Isr: 1,0
    Topic: my-topic Partition: 5    Leader: 0      Replicas: 0,1    Isr: 0,1
    Topic: my-topic Partition: 6    Leader: 1      Replicas: 1,0    Isr: 1,0
    Topic: my-topic Partition: 7    Leader: 0      Replicas: 0,1    Isr: 0,1
#
```

The `--describe` command also has several useful options for filtering the output. These can be helpful for diagnosing cluster issues more easily. For these commands we generally do not specify the `--topic` argument because the intention is to find all topics or partitions in a cluster that match the criteria. These options will not work with the `list` command. Here is a list of useful pairings to use:

```
--topics-with-overrides
```

This will describe only the topics that have configurations that differ from the cluster defaults.

--exclude-internal

The previously mentioned command will remove all topics from the list that begin with the double underscore naming convention.

The following commands are used to help find topic partitions that may have problems:

--under-replicated-partitions

This shows all partitions where one or more of the replicas are not in sync with the leader. This isn't necessarily bad, as cluster maintenance, deployments, and rebalances will cause under-replicated partitions (or URPs) but is something to be aware of.

--at-min-isr-partitions

This shows all partitions where the number of replicas, including the leader, exactly match the setting for minimum in-sync replicas (ISRs). These topics are still available for producer or consumer clients, but all redundancy has been lost, and they are in danger of becoming unavailable.

--under-min-isr-partitions

This shows all partitions where the number of ISRs is below the configured minimum for successful produce actions. These partitions are effectively in read-only mode and cannot be produced to.

--unavailable-partitions

This shows all topic partitions without a leader. This is a serious situation and indicates that the partition is offline and unavailable for producer or consumer clients.

Here's an example of finding topics that are at the minimum ISR settings. In this example, the topic is configured for a min-ISR of 1 and has a replication factor (RF) of 2. Host 0 is online, and host 1 has gone down for maintenance:

```
# kafka-topics.sh --bootstrap-server localhost:9092 --describe --at-min-isr-partitions
      Topic: my-topic Partition: 0      Leader: 0      Replicas: 0,1      Isr: 0
      Topic: my-topic Partition: 1      Leader: 0      Replicas: 0,1      Isr: 0
      Topic: my-topic Partition: 2      Leader: 0      Replicas: 0,1      Isr: 0
      Topic: my-topic Partition: 3      Leader: 0      Replicas: 0,1      Isr: 0
      Topic: my-topic Partition: 4      Leader: 0      Replicas: 0,1      Isr: 0
      Topic: my-topic Partition: 5      Leader: 0      Replicas: 0,1      Isr: 0
      Topic: my-topic Partition: 6      Leader: 0      Replicas: 0,1      Isr: 0
      Topic: my-topic Partition: 7      Leader: 0      Replicas: 0,1      Isr: 0
#
```

Adding Partitions

It is sometimes necessary to increase the number of partitions for a topic. Partitions are the way topics are scaled and replicated across a cluster. The most common reason to increase the partition count is to horizontally scale a topic across more brokers by decreasing the throughput for a single partition. Topics may also be increased if a consumer needs to expand to run more copies in a single consumer group since a partition can only be consumed by a single member in the group.

Following is an example of increasing the number of partitions for a topic named “my-topic” to 16 using the `--alter` command, followed by a verification that it worked:

```
# kafka-topics.sh --bootstrap-server localhost:9092
--alter --topic my-topic --partitions 16

# kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic my-topic
Topic: my-topic PartitionCount: 16      ReplicationFactor: 2      Configs: segment.bytes=1
      Topic: my-topic Partition: 0      Leader: 1      Replicas: 1,0      Isr: 1,0
      Topic: my-topic Partition: 1      Leader: 0      Replicas: 0,1      Isr: 0,1
      Topic: my-topic Partition: 2      Leader: 1      Replicas: 1,0      Isr: 1,0
      Topic: my-topic Partition: 3      Leader: 0      Replicas: 0,1      Isr: 0,1
      Topic: my-topic Partition: 4      Leader: 1      Replicas: 1,0      Isr: 1,0
      Topic: my-topic Partition: 5      Leader: 0      Replicas: 0,1      Isr: 0,1
      Topic: my-topic Partition: 6      Leader: 1      Replicas: 1,0      Isr: 1,0
      Topic: my-topic Partition: 7      Leader: 0      Replicas: 0,1      Isr: 0,1
      Topic: my-topic Partition: 8      Leader: 1      Replicas: 1,0      Isr: 1,0
      Topic: my-topic Partition: 9      Leader: 0      Replicas: 0,1      Isr: 0,1
      Topic: my-topic Partition: 10     Leader: 1      Replicas: 1,0      Isr: 1,0
      Topic: my-topic Partition: 11     Leader: 0      Replicas: 0,1      Isr: 0,1
      Topic: my-topic Partition: 12     Leader: 1      Replicas: 1,0      Isr: 1,0
      Topic: my-topic Partition: 13     Leader: 0      Replicas: 0,1      Isr: 0,1
      Topic: my-topic Partition: 14     Leader: 1      Replicas: 1,0      Isr: 1,0
      Topic: my-topic Partition: 15     Leader: 0      Replicas: 0,1      Isr: 0,1

#
```

ADJUSTING KEYED TOPICS

Topics that are produced with keyed messages can be very difficult to add partitions to from a consumer’s point of view. This is because the mapping of keys to partitions will change when the number of partitions is changed. For this reason, it is advisable to set the number of partitions for a topic that will contain keyed messages once, when the topic is created, and avoid resizing the topic.

Reducing Partitions

It is not possible to reduce the number of partitions for a topic. Deleting a partition from a topic would cause part of the data in that topic to be deleted as well, which would be inconsistent from a client point of view. In addition, trying to redistribute the data to the remaining partitions would be difficult and result in out-of-order messages. Should you need to reduce the number of partitions, it is recommended to delete the topic and re-create it or (if deletion is not possible) create a new version of the existing topic and move all produce traffic to the new topic (e.g., “my-topic-v2”).

Deleting a Topic

Even a topic with no messages uses cluster resources such as disk space, open filehandles, and memory. The controller also has junk metadata that it must retain knowledge of, which can hinder performance at large scale. If a topic is no longer needed, it can be deleted to free up these resources. To perform this action, the brokers in the cluster must be configured with the `delete.topic.enable` option set to `true`. If it’s set to `false`, then the request to delete the topic will be ignored and will not succeed.

Topic deletion is an asynchronous operation. This means that running this command will mark a topic for deletion, but the deletion may not happen immediately, depending on the amount of data and cleanup needed. The controller will notify the brokers of the pending deletion as soon

as possible (after existing controller tasks complete), and the brokers will then invalidate the metadata for the topic and delete the files from disk. It is highly recommended that operators not delete more than one or two topics at a time, and give those ample time to complete before deleting other topics, due to limitations in the way the controller executes these operations. In the small cluster shown in the examples in this book, topic deletion will happen almost immediately, but in larger clusters it may take longer.

DATA LOSS AHEAD

Deleting a topic will also delete all its messages. This is not a reversible operation. Make sure it is executed carefully.

Here is an example of deleting the topic named “my-topic” using the `--delete` argument. Depending on the version of Kafka, there will be a note letting you know that the argument will not work if another config is not set:

```
# kafka-topics.sh --bootstrap-server localhost:9092
--delete --topic my-topic
```

```
Note: This will have no impact if delete.topic.enable is not set
to true.
#
```

You will notice there is no visible feedback that the topic deletion was completed successfully or not. Verify that deletion was successful by running the `--list` or `--describe` options to see that the topic is no longer in the cluster.

Consumer Groups

Consumer groups are coordinated groups of Kafka consumers consuming from topics or multiple partitions of a single topic. The `kafka-consumer-groups.sh` tool helps manage and gain insight into the consumer groups that are consuming from topics in the cluster. It can be used to list consumer groups, describe specific groups, delete consumer groups or specific group info, or reset consumer group offset information.

ZOOKEEPER-BASED CONSUMER GROUPS

In older versions of Kafka, consumer groups could be managed and maintained in ZooKeeper. This behavior was deprecated in versions 0.11.0.* and later, and old consumer groups are no longer used. Some versions of the provided scripts may still show deprecated `--zookeeper` connection string commands, but it is not recommended to use them unless you have an old environment with some consumer groups that have not upgraded to later versions of Kafka.

List and Describe Groups

To list consumer groups, use the `--bootstrap-server` and `--list` parameters. Ad hoc consumers utilizing the `kafka-consumer-groups.sh` script will show up as `console-consumer-<generated_id>` in the consumer list:

```
# kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list
console-consumer-95554
console-consumer-9581
my-consumer
#
```

For any group listed, you can get more details by changing the `--list` parameter to `--describe` and adding the `--group` parameter. This will list all the topics and partitions that the group is consuming from, as well as additional information such as the offsets for each topic partition. [Table 12-1](#) has a full description of all the fields provided in the output.

For example, get consumer group details for the ad hoc group named “my-consumer”:

```
# kafka-consumer-groups.sh --bootstrap-server localhost:9092
--describe --group my-consumer
GROUP          TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG          CONS
my-consumer    my-topic       0          2               4               2            con
my-consumer    my-topic       1          2               3               1            con
my-consumer    my-topic       2          2               3               1            con
#
```

Table 12-1. Fields provided for group named “my-consumer”

Field	Description
GROUP	The name of the consumer group.
TOPIC	The name of the topic being consumed.
PARTITION	The ID number of the partition being consumed.
CURRENT-OFFSET	The next offset to be consumed by the consumer group for this topic partition. This is the position of the consumer within the partition.
LOG-END-OFFSET	The current high-water mark offset from the broker for the topic partition. This is the offset of the next message to be produced to this partition.
LAG	The difference between the consumer Current-Offset and the broker Log-End-Offset for this topic partition.
CONSUMER-ID	A generated unique consumer-id based on the provided client-id.
HOST	Address of the host the consumer group is reading from.
CLIENT-ID	String provided by the client identifying the client that is consuming from the group.

Delete Group

Deletion of consumer groups can be performed with the `--delete` argument. This will remove the entire group, including all stored offsets for all topics that the group is consuming. To perform this action, all consumers

in the group should be shut down as the consumer group must not have any active members. If you attempt to delete a group that is not empty, an error stating “The group is not empty” will be thrown and nothing will happen. It is also possible to use the same command to delete offsets for a single topic that the group is consuming without deleting the entire group by adding the `--topic` argument and specifying which topic offsets to delete.

Here is an example of deleting the entire consumer group named “my-consumer”:

```
# kafka-consumer-groups.sh --bootstrap-server localhost:9092 --delete --group my-consumer
Deletion of requested consumer groups ('my-consumer') was successful.
#
```

Offset Management

In addition to displaying and deleting the offsets for a consumer group, it is also possible to retrieve the offsets and store new offsets in a batch. This is useful for resetting the offsets for a consumer when there is a problem that requires messages to be reread, or for advancing offsets and skipping past a message that the consumer is having a problem with (e.g., if there is a badly formatted message that the consumer cannot handle).

Export offsets

To export offsets from a consumer group to a CSV file, use the `--reset-offsets` argument with the `--dry-run` option. This will allow us to create an export of the current offsets in a file format that can be reused for importing or rolling back the offsets later. The CSV format export will be in the following configuration:

```
<topic-name>,<partition-number>,<offset>
```

Running the same command without the `--dry-run` option will reset the offsets completely, so be careful.

Here is an example of exporting the offsets for the topic “my-topic” that is being consumed by the consumer group named “my-consumer” to a file named *offsets.csv*:

```
# kafka-consumer-groups.sh --bootstrap-server localhost:9092
--export --group my-consumer --topic my-topic
--reset-offsets --to-current --dry-run > offsets.csv

# cat offsets.csv
my-topic,0,8905
my-topic,1,8915
my-topic,2,9845
my-topic,3,8072
my-topic,4,8008
my-topic,5,8319
my-topic,6,8102
my-topic,7,12739
#
```

Import offsets

The import offset tool is the opposite of exporting. It takes the file produced by exporting offsets in the previous section and uses it to set the current offsets for the consumer group. A common practice is to export

the current offsets for the consumer group, make a copy of the file (so that you preserve a backup), and edit the copy to replace the offsets with the desired values.

STOP CONSUMERS FIRST

Before performing this step, it is important that all consumers in the group are stopped. They will not read the new offsets if they are written while the consumer group is active. The consumers will just overwrite the imported offsets.

In the following example, we import the offsets for the consumer group named “my-consumer” from the file we created in the last example named *offsets.csv*:

```
# kafka-consumer-groups.sh --bootstrap-server localhost:9092
--reset-offsets --group my-consumer
--from-file offsets.csv --execute
```

TOPIC	PARTITION	NEW-OFFSET
my-topic	0	8905
my-topic	1	8915
my-topic	2	9845
my-topic	3	8072
my-topic	4	8008
my-topic	5	8319
my-topic	6	8102
my-topic	7	12739

```
#
```

Dynamic Configuration Changes

There is a plethora of configurations for topics, clients, brokers, and more that can be updated dynamically during runtime without having to shut down or redeploy a cluster. The `kafka-configs.sh` is the main tool for modifying these configs. Currently there are four main categories, or *entity-types*, of dynamic config changes that can be made: *topics*, *brokers*, *users*, and *clients*. For each entity-type there are specific configurations that can be overridden. New dynamic configs are being added constantly with each release of Kafka, so it is good to ensure you have the same version of this tool that matches the version of Kafka you are running. For ease of setting up these configs consistently via automation, the `--add-config-file` argument can be used with a preformatted file of all the configs you want to manage and update.

Overriding Topic Configuration Defaults

There are many configurations that are set by default for topics that are defined in the static broker configuration files (e.g., retention time policy). With dynamic configurations, we can override the cluster-level defaults for individual topics to accommodate different use cases within a single cluster. [Table 12-2](#) shows the valid configuration keys for topics that can be altered dynamically.

The format of the command to change a topic configuration is:

```
kafka-configs.sh --bootstrap-server localhost:9092
--alter --entity-type topics --entity-name <topic-name>
--add-config <key>=<value>[,<key>=<value>...]
```

Here is an example of setting the retention for the topic named “my-topic” to 1 hour (3,600,000 ms):

```
# kafka-configs.sh --bootstrap-server localhost:9092
--alter --entity-type topics --entity-name my-topic
--add-config retention.ms=3600000
Updated config for topic: "my-topic".
#
```

Table 12-2. Valid keys for topics

Configuration key	Description
<code>cleanup.policy</code>	If set to <code>compact</code> , the messages in this topic will be discarded and only the most recent message with a given key is retained (log compacted).
<code>compression.type</code>	The compression type used by the broker when writing message batches for this topic to disk.
<code>delete.retention.ms</code>	How long, in milliseconds, deleted tombstones will be retained for this topic. Only valid for log compacted topics.
<code>file.delete.delay.ms</code>	How long, in milliseconds, to wait before deleting log segments and indices for this topic from disk.
<code>flush.messages</code>	How many messages are received before forcing a flush of this topic's messages to disk.
<code>flush.ms</code>	How long, in milliseconds, before forcing a flush of this topic's messages to disk.
<code>follower.replication.throttled.replicas</code>	A list of replicas for which log replication should be throttled by the follower.
<code>index.interval.bytes</code>	How many bytes of messages can be produced between entries in the log segment's index.
<code>leader.replication.throttled.replicas</code>	A list of replicas for which log replication should be throttled by the leader.
<code>max.compaction.lag.ms</code>	Maximum time limit a message won't be eligible for compaction in the log.
<code>max.message.bytes</code>	The maximum size of a single message for this topic, in bytes.
<code>message.downconversion.enable</code>	Allows the message format version to be down-converted to the previous version if enabled with some overhead.
<code>message.format.version</code>	The message format version that the broker will use when writing messages to disk. Must be a valid API version number.
<code>message.timestamp.difference.max.ms</code>	The maximum allowed difference, in milliseconds, between the message timestamp and the broker timestamp when the message is received. This is only valid if the <code>message.timestamp.type</code> is set to <code>CreateTime</code> .

Configuration key	Description
<code>message.timestamp.type</code>	Which timestamp to use when writing messages to disk. Current values are <code>CreateTime</code> for the timestamp specified by the client and <code>LogAppendTime</code> for the time when the message is written to the partition by the broker.
<code>min.cleanable.dirty.ratio</code>	How frequently the log compactor will attempt to compact partitions for this topic, expressed as a ratio of the number of uncompact log segments to the total number of log segments. Only valid for log compacted topics.
<code>min.compaction.lag.ms</code>	Minimum time a message will remain uncompact in the log.
<code>min.insync.replicas</code>	The minimum number of replicas that must be in sync for a partition of the topic to be considered available.
<code>preallocate</code>	If set to <code>true</code> , log segments for this topic should be preallocated when a new segment is rolled.
<code>retention.bytes</code>	The amount of messages, in bytes, to retain for this topic.
<code>retention.ms</code>	How long messages should be retained for this topic, in milliseconds.
<code>segment.bytes</code>	The amount of messages, in bytes, that should be written to a single log segment in a partition.
<code>segment.index.bytes</code>	The maximum size, in bytes, of a single log segment index.
<code>segment.jitter.ms</code>	A maximum number of milliseconds that is randomized and added to <code>segment.ms</code> when rolling log segments.
<code>segment.ms</code>	How frequently, in milliseconds, the log segment for each partition should be rotated.
<code>unclean.leader.election.enable</code>	If set to <code>false</code> , unclean leader elections will not be permitted for this topic.

Overriding Client and User Configuration Defaults

For Kafka clients and users, there are only a few configurations that can be overridden, which are all essentially types of quotas. Two of the more common configurations to change are the bytes/sec rates allowed for producers and consumers with a specified client ID on a per-broker basis. The full list of shared configurations that can be modified for both *users* and *clients* is shown in [Table 12-3](#).

Because throttling occurs on a per-broker basis, even balance of leadership of partitions across a cluster becomes particularly important to enforce this properly. If you have 5 brokers in a cluster and you specify a producer quota of 10 MBps for a client, that client will be allowed to produce 10 MBps *on each* broker at the same time for a total of 50 MBps, assuming a balanced leadership across all 5 hosts. However, if leadership for every partition is all on broker 1, the same producer will only be able to produce a max of 10 MBps.

Table 12-3. The configurations (keys) for clients

Configuration key	Description
<code>consumer_byte_s_rate</code>	The amount of messages, in bytes, that a single client ID is allowed to consume from a single broker in one second.
<code>producer_byte_s_rate</code>	The amount of messages, in bytes, that a single client ID is allowed to produce to a single broker in one second.
<code>controller_mutations_rate</code>	The rate at which mutations are accepted for the create topics request, the create partitions request, and the delete topics request. The rate is accumulated by the number of partitions created or deleted.
<code>request_percentage</code>	The percentage per quota window (out of a total of $(\text{num.io.threads} + \text{num.network.threads}) \times 100\%$) for requests from the user or client.

CLIENT ID VERSUS CONSUMER GROUP

The client ID is not necessarily the same as the consumer group name. Consumers can set their own client ID, and you may have many consumers that are in different groups that specify the same client ID. It is considered a best practice to set the client ID for each consumer group to something unique that identifies that group. This allows a single consumer group to share a quota, and it makes it easier to identify in logs what group is responsible for requests.

Compatible user and client config changes can be specified together for compatible configs that apply to both. Here is an example of the command to change the controller mutation rate for both a user and client in one configuration step:

```
# kafka-configs.sh --bootstrap-server localhost:9092
--alter --add-config "controller_mutations_rate=10"
--entity-type clients --entity-name <client ID>
--entity-type users --entity-name <user ID>
#
```

Overriding Broker Configuration Defaults

Broker- and cluster-level configs will primarily be configured statically in the cluster configuration files, but there is a plethora of configs that can be overridden during runtime without needing to redeploy Kafka. More than 80 overrides can be altered with *kafka-configs.sh* for brokers. As

such, we will not list them all in this book, but they can be referenced by the `--help` command or found in the [open source documentation](#). A few important configs worth pointing out specifically are:

min.insync.replicas

Adjusts the minimum number of replicas that need to acknowledge a write for a produce request to be successful when producers have set acks to `all` (or `-1`).

unclean.leader.election.enable

Allows replicas to be elected as leader even if it results in data loss. This is useful when it is permissible to have some lossy data, or to turn on for short times to unstick a Kafka cluster if unrecoverable data loss cannot be avoided.

max.connections

The maximum number of connections allowed to a broker at any time. We can also use `max.connections.per.ip` and `max.connections.per.ip.overrides` for more fine-tuned throttling.

Describing Configuration Overrides

All configuration overrides can be listed using the `kafka-config.sh` tool. This will allow you to examine the specific configuration for a topic, broker, or client. Similar to other tools, this is done using the `--describe` command.

In the following example, we can get all the configuration overrides for the topic named “my-topic,” which we observe is only the retention time:

```
# kafka-configs.sh --bootstrap-server localhost:9092
--describe --entity-type topics --entity-name my-topic
Configs for topics:my-topic are
retention.ms=3600000
#
```

TOPIC OVERRIDES ONLY

The configuration description will only show overrides—it does not include the cluster default configurations. There is not a way to dynamically discover the configuration of the brokers themselves. This means that when using this tool to discover topic or client settings in automation, the user must have separate knowledge of the cluster default configuration.

Removing Configuration Overrides

Dynamic configurations can be removed entirely, which will cause the entity to revert back to the cluster defaults. To delete a configuration override, use the `--alter` command along with the `--delete-config` parameter.

For example, delete a configuration override for `retention.ms` for a topic named “my-topic”:

```
# kafka-configs.sh --bootstrap-server localhost:9092
--alter --entity-type topics --entity-name my-topic
--delete-config retention.ms
```

```
Updated config for topic: "my-topic".  
#
```

Producing and Consuming

While working with Kafka, you will often find it is necessary to manually produce or consume some sample messages in order to validate what's going on with your applications. Two utilities are provided to help with this, `kafka-console-consumer.sh` and `kafka-console-producer.sh`, which were touched upon briefly in [Chapter 2](#) to verify our installation. These tools are wrappers around the main Java client libraries that allow you to interact with Kafka topics without having to write an entire application to do it.

PIPING OUTPUT TO ANOTHER APPLICATION

While it is possible to write applications that wrap around the console consumer or producer (e.g., to consume messages and pipe them to another application for processing), this type of application is quite fragile and should be avoided. It is difficult to interact with the console consumer in a way that does not lose messages. Likewise, the console producer does not allow for using all features, and properly sending bytes is tricky. It is best to use either the Java client libraries directly or a third-party client library for other languages that use the Kafka protocol directly.

Console Producer

The `kafka-console-producer.sh` tool can be used to write messages into a Kafka topic in your cluster. By default, messages are read one per line, with a tab character separating the key and the value (if no tab character is present, the key is null). As with the console consumer, the producer reads in and produces raw bytes using the default serializer (which is `DefaultEncoder`).

The console producer requires that a minimum of two arguments are provided to know what Kafka cluster to connect to and which topic to produce to within that cluster. The first is the customary `--bootstrap-server` connection string we are used to using. When you are done producing, send an end-of-file (EOF) character to close the client. In most common terminals, this is done with Control-D.

Here we can see an example of producing four messages to a topic named “my-topic”:

```
# kafka-console-producer.sh --bootstrap-server localhost:9092 --topic my-topic  
>Message 1  
>Test Message 2  
>Test Message 3  
>Message 4  
>^D  
#
```

Using producer configuration options

It is possible to pass normal producer configuration options to the console producer as well. This can be done in two ways, depending on how many options you need to pass and how you prefer to do it. The first is to provide a producer configuration file by specifying `--producer.config <config-file>`, where `<config-file>` is the full path to a file that

contains the configuration options. The other way is to specify the options on the command line with one or more arguments of the form `--producer-property <key> = <value>`, where `<key>` is the configuration option name and `<value>` is the value to set it to. This can be useful for producer options like message-batching configurations (such as `linger.ms` or `batch.size`).

CONFUSING COMMAND-LINE OPTIONS

The `--property` command-line option is available for both the console producer and the console consumer, but this should not be confused with the `--producer-property` or `--consumer-property` options, respectively. The `--property` option is only used for passing configurations to the message formatter, and not the client itself.

The console producer has many command-line arguments available to use with the `--producer-property` option for adjusting its behavior. Some of the more useful options are:

`--batch-size`

Specifies the number of messages sent in a single batch if they are not being sent synchronously.

`--timeout`

If a producer is running in asynchronous mode, this provides the max amount of time waiting for the batch size before producing to avoid long waits on low-producing topics.

`--compression-codec <string>`

Specify the type of compression to be used when producing messages. Valid types can be one of the following: `none`, `gzip`, `snappy`, `zstd`, or `lz4`. The default value is `gzip`.

`--sync`

Produce messages synchronously, waiting for each message to be acknowledged before sending the next one.

Line-reader options

The `kafka.tools.ConsoleProducer$LineMessageReader` class, which is responsible for reading standard input and creating producer records, also has several useful options that can be passed to the console producer using the `--property` command-line option:

`ignore.error`

Set to `false` to throw an exception when `parse.key` is set to `true` and a key separator is not present. Defaults to `true`.

`parse.key`

Set to `false` to always set the key to null. Defaults to `true`.

`key.separator`

Specify the delimiter character to use between the message key and message value when reading. Defaults to a tab character.

CHANGING LINE-READING BEHAVIOR

You can provide your own class to Kafka for customized methods of reading lines. The class that you create must extend `kafka.common.MessageReader` and will be responsible for creating the `ProducerRecord`. Specify your class on the command line with the `--line-reader` option, and make sure the JAR containing your class is in the classpath. The default is `kafka.tools.ConsoleProducer$LineMessageReader`.

When producing messages, the `LineMessageReader` will split the input on the first instance of the `key.separator`. If there are no characters remaining after that, the value of the message will be empty. If no key separator character is present on the line, or if `parse.key` is false, the key will be null.

Console Consumer

The `kafka-console-consumer.sh` tool provides a means to consume messages out of one or more topics in your Kafka cluster. The messages are printed in standard output, delimited by a new line. By default, it outputs the raw bytes in the message, without the key, with no formatting (using the `DefaultFormatter`). Similar to the producer, there are a few basic options needed to get started: a connection string to the cluster, which topic you want to consume from, and the timeframe you want to consume.

CHECKING TOOL VERSIONS

It is very important to use a consumer that is the same version as your Kafka cluster. Older console consumers can potentially damage the cluster by interacting with the cluster or ZooKeeper in incorrect ways.

As in other commands, the connection string to the cluster will be the `--bootstrap-server` option; however, you can choose from two options for selecting the topics to consume:

`--topic`

Specifies a single topic to consume from.

`--whitelist`

A regular expression matching all topics to consume from (remember to properly escape the regex so that it is not processed improperly by the shell).

Only one of the previous options should be selected and used. Once the console consumer has started, the tool will continue to try and consume until the shell escape command is given (in this case, Ctrl-C). Here is an example of consuming all topics from our cluster that match the prefix `my` (of which there is only one in this example, “my-topic”):

```
# kafka-console-consumer.sh --bootstrap-server localhost:9092
--whitelist 'my.*' --from-beginning
Message 1
Test Message 2
Test Message 3
Message 4
^C
#
```

Using consumer configuration options

In addition to these basic command-line options, it is possible to pass normal consumer configuration options to the console consumer as well. Similar to the `kafka-console-producer.sh` tool, this can be done in two ways, depending on how many options you need to pass and how you prefer to do it. The first is to provide a consumer configuration file by specifying `--consumer.config <config-file>`, where `<config-file>` is the full path to a file that contains the configuration options. The other way is to specify the options on the command line with one or more arguments of the form `--consumer-property <key> = <value>`, where `<key>` is the configuration option name and `<value>` is the value to set it to.

There are a few other commonly used options for the console consumer that are helpful to know and be familiar with:

`--formatter <classname>`

Specifies a message formatter class to be used to decode the messages. This defaults to `kafka.tools.DefaultMessageFormatter`.

`--from-beginning`

Consume messages in the topic(s) specified from the oldest offset. Otherwise, consumption starts from the latest offset.

`--max-messages <int>`

The maximum number of messages to consume before exiting.

`--partition <int>`

Consume only from the partition with the ID given.

`--offset`

The offset ID to consume from, if provided (`<int>`). Other valid options are `earliest`, which will consume from the beginning, and `latest`, which will start consuming from the most recent offset.

`--skip-message-on-error`

Skip a message if there is an error when processing instead of halting. Useful for debugging.

Message formatter options

There are three message formatters available to use besides the default:

`kafka.tools.LoggingMessageFormatter`

Outputs messages using the logger, rather than standard out. Messages are printed at the INFO level and include the timestamp, key, and value.

`kafka.tools.ChecksumMessageFormatter`

Prints only message checksums.

`kafka.tools.NoOpMessageFormatter`

Consumes messages but does not output them at all.

The following is an example of consuming the same messages from before but with the `kafka.tools.ChecksumMessageFormatter` being used rather than the default:

```
# kafka-console-consumer.sh --bootstrap-server localhost:9092
--whitelist 'my.*' --from-beginning
--formatter kafka.tools.ChecksumMessageFormatter
checksum:0
checksum:0
checksum:0
checksum:0
#
```

The `kafka.tools.DefaultMessageFormatter` also has several useful options that can be passed using the `--property` command-line option, shown in [Table 12-4](#).

Table 12-4. Message formatter properties

Property	Description
<code>print.timestamp</code>	Set to <code>true</code> to display the timestamp of each message (if available).
<code>print.key</code>	Set to <code>true</code> to display the message key in addition to the value.
<code>print.offset</code>	Set to <code>true</code> to display the message offset in addition to the value.
<code>print.partition</code>	Set to <code>true</code> to display the topic partition a message is consumed from.
<code>key.separator</code>	Specify the delimiter character to use between the message key and message value when printing.
<code>line.separator</code>	Specify the delimiter character to use between messages.
<code>key.deserializer</code>	Provide a class name that is used to deserialize the message key before printing.
<code>value.deserializer</code>	Provide a class name that is used to deserialize the message value before printing.

The deserializer classes must implement `org.apache.kafka.common.serialization.Deserializer`, and the console consumer will call the `toString` method on them to get the output to display. Typically, you would implement these deserializers as a Java class that you would insert into the classpath for the console consumer by setting the `CLASSPATH` environment variable before executing `kafka_console_consumer.sh`.

Consuming the offsets topics

It is sometimes useful to see what offsets are being committed for the cluster's consumer groups. You may want to see if a particular group is committing offsets at all, or how often offsets are being committed. This can be done by using the console consumer to consume the special internal topic called `__consumer_offsets`. All consumer offsets are written as messages to this topic. In order to decode the messages in this topic,

you must use the formatter class `kafka.coordinator.group.GroupMetadataManager$OffsetsMessageFormatter`.

Putting all we have learned together, the following is an example of consuming the earliest message from the `__consumer_offsets` topic:

```
# kafka-console-consumer.sh --bootstrap-server localhost:9092
--topic __consumer_offsets --from-beginning --max-messages 1
--formatter "kafka.coordinator.group.GroupMetadataManager$OffsetsMessageFormatter"
--consumer-property exclude.internal.topics=false
[my-group-name,my-topic,0]::[OffsetMetadata[1,NO_METADATA]
CommitTime 1623034799990 ExpirationTime 1623639599990]
Processed a total of 1 messages
#
```

Partition Management

A default Kafka installation also contains a few scripts for working with the management of partitions. One of these tools allows for the reelection of leader replicas; another is a low-level utility for assigning partitions to brokers. Together these tools can assist in situations where a more manual hands-on approach to balance message traffic within a cluster of Kafka brokers is needed.

Preferred Replica Election

As described in [Chapter 7](#), partitions can have multiple replicas for reliability. It is important to understand that only one of these replicas can be the leader for the partition at any given point in time, and all produce and consume operations happen on that broker. Maintaining a balance of which partition's replicas have leadership on which broker is necessary to ensure the load is spread out through a full Kafka cluster.

Leadership is defined within Kafka as the first in-sync replica in the replica list. However, when a broker is stopped or loses connectivity to the rest of the cluster, leadership is transferred to another in-sync replica, and the original does not resume leadership of any partitions automatically. This can cause wildly inefficient balance after a deployment across a full cluster if automatic leader balancing is not enabled. As such it is recommended to ensure that this setting is enabled or to use other open source tooling such as Cruise Control to ensure that a good balance is maintained at all times.

If you find that your Kafka cluster has a poor balance, a lightweight, generally non-impacting procedure can be performed called *preferred leader election*. This tells the cluster controller to select the ideal leader for partitions. Clients can track leadership changes automatically, so they will be able to move to the new broker in the cluster in which leadership is transferred. This operation can be manually triggered using the `kafka-leader-election.sh` utility. An older version of this tool called `kafka-preferred-replica-election.sh` is also available but has been deprecated in favor of the new tool, which allows for more customization, such as specifying whether we want a “preferred” or “unclean” election type.

As an example, starting a preferred leader election for all topics in a cluster can be executed with the following command:

```
# kafka-leader-election.sh --bootstrap-server localhost:9092
--election-type PREFERRED --all-topic-partitions
```

```
#
```

It is also possible to start elections on specific partitions or topics. This can be done by passing in a topic name with the `--topic` option and a partition with the `--partition` option directly. It is also possible to pass in a list of several partitions to be elected. This is done by configuring a JSON file that we will call *partitions.json*:

```
{
  "partitions": [
    {
      "partition": 1,
      "topic": "my-topic"
    },
    {
      "partition": 2,
      "topic": "foo"
    }
  ]
}
```

In this example, we will start a preferred replica election with a specified list of partitions in a file named *partitions.json*:

```
# kafka-leader-election.sh --bootstrap-server localhost:9092
--election-type PREFERRED --path-to-json-file partitions.json
#
```

Changing a Partition's Replicas

Occasionally it may be necessary to change the replica assignments manually for a partition. Some examples of when this might be needed are:

- There is an uneven load on brokers that the automatic leader distribution is not correctly handling.
- If a broker is taken offline and the partition is under replicated.
- If a new broker is added and we want to more quickly balance new partitions on it.
- You want to adjust the replication factor of a topic.

The `kafka-reassign-partitions.sh` can be used to perform this operation. This is a multistep process to generate a move set and then execute on the provided move set proposal. First, we want to use a broker list and a topic list to generate a proposal for the set of moves. This will require the generation of a JSON file with a list of topics to be supplied. The next step executes the moves that were generated by the previous proposal. Finally, the tool can be used with the generated list to track and verify the progress or completion of the partition reassignments.

Let's generate a hypothetical scenario in which you have a four-broker Kafka cluster. You've recently added two new brokers, bringing the total up to six, and you want to move two of your topics onto brokers 5 and 6.

To generate a set of partition moves, you must first create a file that contains a JSON object listing the topics. The JSON object is formatted as follows (the version number is currently always 1):

```
{
  "topics": [
    {
```

```

        "topic": "foo1"
    },
    {
        "topic": "foo2"
    }
],
"version": 1
}

```

Once we've defined our JSON file, we can use it to generate a set of partition moves to move the topics listed in the file *topics.json* to the brokers with IDs 5 and 6:

```

# kafka-reassign-partitions.sh --bootstrap-server localhost:9092
--topics-to-move-json-file topics.json
--broker-list 5,6 --generate
{
  "version": 1,
  "partitions": [
    { "topic": "foo1", "partition": 2, "replicas": [1, 2] },
    { "topic": "foo1", "partition": 0, "replicas": [3, 4] },
    { "topic": "foo2", "partition": 2, "replicas": [1, 2] },
    { "topic": "foo2", "partition": 0, "replicas": [3, 4] },
    { "topic": "foo1", "partition": 1, "replicas": [2, 3] },
    { "topic": "foo2", "partition": 1, "replicas": [2, 3] }
  ]
}

```

Proposed partition reassignment configuration

```

{
  "version": 1,
  "partitions": [
    { "topic": "foo1", "partition": 2, "replicas": [5, 6] },
    { "topic": "foo1", "partition": 0, "replicas": [5, 6] },
    { "topic": "foo2", "partition": 2, "replicas": [5, 6] },
    { "topic": "foo2", "partition": 0, "replicas": [5, 6] },
    { "topic": "foo1", "partition": 1, "replicas": [5, 6] },
    { "topic": "foo2", "partition": 1, "replicas": [5, 6] }
  ]
}
#

```

The output proposed here is formatted correctly, to which we can save two new JSON files that we will call *revert-reassignment.json* and *expand-cluster-reassignment.json*. The first file can be used to move partitions back to where they were originally if you need to roll back for some reason. The second file can be used for the next step, as this is just a proposal and hasn't executed anything yet. You'll notice in the output that there isn't a good balance of leadership, as the proposal will result in all leadership moving to broker 5. We will ignore this for now and presume the cluster automatic leadership balancing is enabled, which will help distribute it later. It should be noted that the first step can be skipped if you know exactly where you want to move your partitions to and you manually craft the JSON to move partitions.

To execute the proposed partition reassignment from the file *expand-cluster-reassignment.json*, run the following command:

```

# kafka-reassign-partitions.sh --bootstrap-server localhost:9092
--reassignment-json-file expand-cluster-reassignment.json
--execute
Current partition replica assignment

{
  "version": 1,
  "partitions": [
    { "topic": "foo1", "partition": 2, "replicas": [1, 2] },
    { "topic": "foo1", "partition": 0, "replicas": [3, 4] },
    { "topic": "foo2", "partition": 2, "replicas": [1, 2] },
    { "topic": "foo2", "partition": 0, "replicas": [3, 4] },
    { "topic": "foo1", "partition": 1, "replicas": [2, 3] },

```

```

        {"topic":"foo2","partition":1,"replicas":[2,3]}}
    }

    Save this to use as the --reassignment-json-file option during rollback
    Successfully started reassignment of partitions
    {"version":1,
    "partitions":[{"topic":"foo1","partition":2,"replicas":[5,6]},
                  {"topic":"foo1","partition":0,"replicas":[5,6]},
                  {"topic":"foo2","partition":2,"replicas":[5,6]},
                  {"topic":"foo2","partition":0,"replicas":[5,6]},
                  {"topic":"foo1","partition":1,"replicas":[5,6]},
                  {"topic":"foo2","partition":1,"replicas":[5,6]}]
    }
#

```

This will start the reassignment of the specified partition replicas to the new brokers. The output is the same as the generated proposal verification. The cluster controller performs this reassignment action by adding the new replicas to the replica list for each partition, which will temporarily increase the replication factor of these topics. The new replicas will then copy all existing messages for each partition from the current leader. Depending on the size of the partitions on disk, this can take a significant amount of time as the data is copied across the network to the new replicas. Once replication is complete, the controller removes the old replicas from the replica list by reducing the replication factor to the original size with the old replicas removed.

Here are a few other useful features of the command you could take advantage of:

--additional

This option will allow you to add to the existing reassignments so they can continue to be performed without interruption and without the need to wait until the original movements have completed in order to start a new batch.

--disable-rack-aware

There may be times when, due to rack awareness settings, the end-state of a proposal may not be possible. This can be overridden with this command if necessary.

--throttle

This value is in units of bytes/sec. Partition reassignments have a big impact on the performance of your cluster, as they will cause changes in the consistency of the memory page cache and use network and disk I/O. Throttling the movement of partitions can be useful to prevent this issue. This can be combined with the *--additional* tag to throttle an already-started reassignment process that may be causing issues.

When removing many partitions from a single broker, such as if that broker is being removed from the cluster, it may be useful to remove all leadership from the broker first. This can be done by manually moving leaderships off the broker; however, using the preceding tooling to do this is arduous. Other open source tools such as Cruise Control include features like broker “demotion,” which safely moves leadership off a broker and is probably the simplest way to do this.

However, if you do not have access to such tools, a simple restart of a broker will suffice. As a broker is preparing to shut down, all leadership for the partitions on that particular broker will move to other brokers in the clusters. This can significantly increase the performance of reassignments and reduce the impact on the cluster, as the replication traffic will be distributed to many brokers. However, if automatic leader reassignment is enabled after the broker is bounced, leadership may return to this broker, so it may be beneficial to temporarily disable this feature.

To check on the progress of the partition moves, the tool can be used to verify the status of the reassignment. This will show which reassignments are currently in progress, which reassignments have completed, and (if there was an error) which reassignments have failed. To do this, you must have the file with the JSON object that was used in the execute step.

Here is an example of potential results using the `--verify` option when running the preceding partition reassignment from the file *expand-cluster-reassignment.json*:

```
# kafka-reassign-partitions.sh --bootstrap-server localhost:9092
--reassignment-json-file expand-cluster-reassignment.json
--verify
Status of partition reassignment:
  Status of partition reassignment:
    Reassignment of partition [foo1,0] completed successfully
    Reassignment of partition [foo1,1] is in progress
    Reassignment of partition [foo1,2] is in progress
    Reassignment of partition [foo2,0] completed successfully
    Reassignment of partition [foo2,1] completed successfully
    Reassignment of partition [foo2,2] completed successfully
#
```

Changing the replication factor

The `kafka-reassign-partitions.sh` tool can also be used to increase or decrease the replication factor (RF) for a partition. This may be necessary in situations where a partition was created with the wrong RF, you want increased redundancy as you expand your cluster, or you want to decrease redundancy for cost savings. One clear example is that if a cluster RF default setting is adjusted, existing topics will not automatically be increased. The tool can be used to increase RF on the existing partitions.

As an example, if we wanted to increase topic “foo1” from the previous example from an RF = 2 to RF = 3, then we could craft a JSON similar to the execution proposal we used before, except we’d add in an additional broker ID to the replica set. For example, we could construct a JSON called *increase-foo1-RF.json* in which we add broker 4 to the existing set of 5,6 that we already have:

```
{
  { "version": 1,
    "partitions": [{ "topic": "foo1", "partition": 1, "replicas": [5,6,4]},
                  { "topic": "foo1", "partition": 2, "replicas": [5,6,4]},
```



```

    {"topic": "foo1", "partition": 3, "replicas": [5, 6, 4]},
  }
}

```

We'd then use the commands shown earlier to execute on this proposal. When it completes, we can verify the RF has been increased by either using the `--verify` flag or using the `kafka-topics.sh` script to describe the topic:

```

# kafka-topics.sh --bootstrap-server localhost:9092 --topic foo1 --describe
Topic:foo1      PartitionCount:3      ReplicationFactor:3      Configs:
  Topic: foo1 Partition: 0    Leader: 5      Replicas: 5,6,4 Isr: 5,6,4
  Topic: foo1 Partition: 1    Leader: 5      Replicas: 5,6,4 Isr: 5,6,4
  Topic: foo1 Partition: 2    Leader: 5      Replicas: 5,6,4 Isr: 5,6,4
#

```

Canceling replica reassignments

Canceling a replica reassignment in the past was a dangerous process that required unsafe manual manipulation of ZooKeeper nodes (or znodes) by deleting the `/admin/reassign_partitions` znode. Fortunately, this is no longer the case. The `kafka-reassign-partitions.sh` script (as well as the AdminClient it is a wrapper for) now supports the `--cancel` option, which will cancel the active reassignments that are ongoing in a cluster. When stopping an in-progress partition move, the `--cancel` command is designed to restore the replica set to the one it was prior to reassignment being initiated. As such, if replicas are being removed from a dead broker or an overloaded broker, it may leave the cluster in an undesirable state. There is also no guarantee that the reverted replica set will be in the same order as it was previously.

Dumping Log Segments

On occasion you may have the need to read the specific content of a message, perhaps because you ended up with a “poison pill” message in your topic that is corrupted and your consumer cannot handle it. The `kafka-dump-log.sh` tool is provided to decode the log segments for a partition. This will allow you to view individual messages without needing to consume and decode them. The tool takes a comma-separated list of log segment files as an argument and can print out either message summary information or detailed message data.

In this example, we will dump the logs from a sample topic, “my-topic,” which is a new topic with only four messages in it. First, we will simply decode the log segment file named `00000000000000000000.log` and retrieve basic metadata info about each message without actually printing the message contents. In our example Kafka installation, the Kafka data directory is set up in `/tmp/kafka-logs`. As such, our directory for finding the log segments will be `/tmp/kafka-logs/<topic-name>-<partition>`, in this case, `/tmp/kafka-logs/my-topic-0/`:

```

# kafka-dump-log.sh --files /tmp/kafka-logs/my-topic-0/00000000000000000000.log
Dumping /tmp/kafka-logs/my-topic-0/00000000000000000000.log
Starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: -1 lastSequence: -1
  producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0
  isTransactional: false isControl: false position: 0
  CreateTime: 1623034799990 size: 77 magic: 2
  compresscodec: NONE crc: 1773642166 invalid: true

```

```

baseOffset: 1 lastOffset: 1 count: 1 baseSequence: -1 lastSequence: -1
  producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0
  isTransactional: false isControl: false position: 77
  CreateTime: 1623034803631 size: 82 magic: 2
  compresscodec: NONE crc: 1638234280 invalid: true
baseOffset: 2 lastOffset: 2 count: 1 baseSequence: -1 lastSequence: -1
  producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0
  isTransactional: false isControl: false position: 159
  CreateTime: 1623034808233 size: 82 magic: 2
  compresscodec: NONE crc: 4143814684 invalid: true
baseOffset: 3 lastOffset: 3 count: 1 baseSequence: -1 lastSequence: -1
  producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0
  isTransactional: false isControl: false position: 241
  CreateTime: 1623034811837 size: 77 magic: 2
  compresscodec: NONE crc: 3096928182 invalid: true
#

```

In the next example, we add the `--print-data-log` option, which will provide us the actual payload information and more:

```

# kafka-dump-log.sh --files /tmp/kafka-logs/my-topic-0/00000000000000000000.log --print-
Dumping /tmp/kafka-logs/my-topic-0/00000000000000000000.log
Starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: -1 lastSequence: -1
  producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0
  isTransactional: false isControl: false position: 0
  CreateTime: 1623034799990 size: 77 magic: 2
  compresscodec: NONE crc: 1773642166 invalid: true
| offset: 0 CreateTime: 1623034799990 keysize: -1 valuesize: 9
  sequence: -1 headerKeys: [] payload: Message 1
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: -1 lastSequence: -1
  producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0
  isTransactional: false isControl: false position: 77
  CreateTime: 1623034803631 size: 82 magic: 2
  compresscodec: NONE crc: 1638234280 invalid: true
| offset: 1 CreateTime: 1623034803631 keysize: -1 valuesize: 14
  sequence: -1 headerKeys: [] payload: Test Message 2
baseOffset: 2 lastOffset: 2 count: 1 baseSequence: -1 lastSequence: -1
  producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0
  isTransactional: false isControl: false position: 159
  CreateTime: 1623034808233 size: 82 magic: 2
  compresscodec: NONE crc: 4143814684 invalid: true
| offset: 2 CreateTime: 1623034808233 keysize: -1 valuesize: 14
  sequence: -1 headerKeys: [] payload: Test Message 3
baseOffset: 3 lastOffset: 3 count: 1 baseSequence: -1 lastSequence: -1
  producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0
  isTransactional: false isControl: false position: 241
  CreateTime: 1623034811837 size: 77 magic: 2
  compresscodec: NONE crc: 3096928182 invalid: true
| offset: 3 CreateTime: 1623034811837 keysize: -1 valuesize: 9
  sequence: -1 headerKeys: [] payload: Message 4
#

```

The tool also contains a few other useful options, such as validating the index file that goes along with a log segment. The index is used for finding messages within a log segment, and if corrupted, will cause errors in consumption. Validation is performed whenever a broker starts up in an unclean state (i.e., it was not stopped normally), but it can be performed manually as well. There are two options for checking indices, depending on how much checking you want to do. The option `--index-sanity-check` will just check that the index is in a usable state, while `--verify-index-only` will check for mismatches in the index without printing out all the index entries. Another useful option, `--value-decoder-class`, allows serialized messages to be deserialized by passing in a decoder.

Replica Verification

Partition replication works similar to a regular Kafka consumer client: the follower broker starts replicating at the oldest offset and checkpoints the current offset to disk periodically. When replication stops and restarts, it picks up from the last checkpoint. It is possible for previously replicated log segments to get deleted from a broker, and the follower will not fill in the gaps in this case.

To validate that the replicas for a topic's partitions are the same across the cluster, you can use the `kafka-replica-verification.sh` tool for verification. This tool will fetch messages from all the replicas for a given set of topic partitions, check that all messages exist on all replicas, and print out the max lag for given partitions. This process will operate continuously in a loop until canceled. To do this, you must provide an explicit comma-separated list of brokers to connect to. By default, all topics are validated; however, you may also provide the tool a regular expression that matches the topics you wish to validate.

CAUTION: CLUSTER IMPACT AHEAD

The replica verification tool will have an impact on your cluster similar to reassigning partitions, as it must read all messages from the oldest offset in order to verify the replica. In addition, it reads from all replicas for a partition in parallel, so it should be used with caution.

For example, verify the replicas for the topics starting with *my* on kafka brokers 1 and 2, which contain partition 0 of “my-topic”:

```
# kafka-replica-verification.sh --broker-list kafka.host1.domain.com:9092,kafka.host2.do
--topic-white-list 'my.*'

2021-06-07 03:28:21,829: verification process is started.
2021-06-07 03:28:51,949: max lag is 0 for partition my-topic-0 at offset 4 among 1 parti
2021-06-07 03:29:22,039: max lag is 0 for partition my-topic-0 at offset 4 among 1 parti
...
#
```

Other Tools

Several more tools are included in the Kafka distribution that are not covered in depth in this book that can be useful in administering your Kafka cluster for specific use cases. Further information about them can be found on the [Apache Kafka website](#):

Client ACLs

A command-line tool, `kafka-acls.sh`, is provided for interacting with access controls for Kafka clients. This includes full features for authorizer properties, set up for deny or allow principles, cluster- or topic-level restrictions, ZooKeeper TLS file configuration, and much more.

Lightweight MirrorMaker

A lightweight `kafka-mirror-maker.sh` script is available for mirroring data. A more in-depth look at replication can be found in [Chapter 10](#).

Testing tools

There are several other scripts used for testing Kafka or helping to perform upgrades of features. `kafka-broker-api-versions.sh` helps to easily identify different versions of usable API elements when upgrading from one Kafka version to another and check for compatibility issues. There are producer and consumer performance tests scripts. There are several scripts to help administer ZooKeeper as well. There is also `trogdor.sh`, which is a test framework designed to run benchmarks and other workloads to attempt to stress test the system.

Unsafe Operations

There are some administrative tasks that are technically possible to do but should not be attempted except in the most extreme situations. Often this is when you are diagnosing a problem and have run out of options, or you have found a specific bug that you need to work around temporarily. These tasks are usually undocumented, unsupported, and pose some amount of risk to your application.

Several of the more common of these tasks are documented here so that in an emergency situation, there is a potential option for recovery. Their use is not recommended under normal cluster operations and should be considered carefully before being executed.

⚠️ DANGER: HERE BE DRAGONS

The operations in this section often involve working with the cluster metadata stored in ZooKeeper directly. This can be a very dangerous operation, so you must be very careful to not modify the information in ZooKeeper directly, except as noted.

Moving the Cluster Controller

Every Kafka cluster has a single broker that is designated as a controller. The controller has a special thread that is responsible for overseeing cluster operations in addition to normal broker work. Normally, controller election is done automatically through ephemeral ZooKeeper znode monitoring. When a controller turns off or becomes unavailable, other brokers nominate themselves as soon as possible, since once the controller shuts down, the znode is removed.

On occasion, when troubleshooting a misbehaving cluster or broker, it may be useful to forcibly move the controller to a different broker without shutting down the host. One such example is when the controller has suffered an exception or other problem that has left it running but not functional. Moving the controller in these situations does not normally have a high risk, but as it is not a normal task, it should not be performed regularly.

To forcibly move a controller, deleting the ZooKeeper znode at `/admin/controller` manually will cause the current controller to resign, and the cluster will randomly select a new controller. There is currently no way to specify a specific broker to be controller in Apache Kafka.

Removing Topics to Be Deleted

When attempting to delete a topic in Kafka, a ZooKeeper node requests that the deletion is created. Once every replica completes deletion of the

topic and acknowledges deletion is complete, the znode will be removed. Under normal circumstances, this is executed by the cluster very quickly. However, sometimes things can go wrong with this process. Here are some scenarios in which a deletion request may become stuck:

1. A requester has no way of knowing whether topic deletion is enabled in the cluster and can request deletion of a topic from a cluster in which deletion is disabled.
2. A very large topic is requested to be deleted, but before the request is handled, one or more of the replica sets goes offline due to hardware failures, and the deletion cannot complete as the controller cannot ack that the deletion was completed successfully.

To “unstick” topic deletion, first delete the `/admin/delete_topic/<topic>` znode. Deleting the topic ZooKeeper nodes (but not the parent `/admin/delete_topic` node) will remove the pending requests. If the deletion is re-queued by cached requests in the controller, it may be necessary to also forcibly move the controller as shown earlier immediately after removing the topic znode to ensure that no cached requests are pending in the controller.

Deleting Topics Manually

If you are running a cluster with delete topics disabled, or if you find yourself needing to delete some topics outside of the normal flow of operations, it is possible to manually delete them from the cluster. This requires a full shutdown of all brokers in the cluster, however, and cannot be done while any of the brokers in the cluster are running.

SHUT DOWN BROKERS FIRST

Modifying the cluster metadata in ZooKeeper when the cluster is online is a very dangerous operation and can put the cluster into an unstable state. Never attempt to delete or modify topic metadata in ZooKeeper while the cluster is online.

To delete a topic from the cluster:

1. Shut down all brokers in the cluster.
2. Remove the ZooKeeper path `/brokers/topics/<topic>` from the Kafka cluster path. Note that this node has child nodes that must be deleted first.
3. Remove the partition directories from the log directories on each broker. These will be named `<topic>-<int>`, where `<int>` is the partition ID.
4. Restart all brokers.

Summary

Running a Kafka cluster can be a daunting endeavor, with numerous configurations and maintenance tasks to keep the systems running at peak performance. In this chapter, we discussed many of the routine tasks, such as managing topic and client configurations, that you will need to handle frequently. We also covered some of the more esoteric tasks that you'll need for debugging problems, like examining log segments. Finally, we covered a few of the operations that, while not safe or routine, can be used to get you out of a sticky situation. All together, these tools will help you to manage your Kafka cluster. As you begin to scale your Kafka clusters larger, even the use of these tools may become arduous and difficult

to manage. It is highly recommended to engage with the open source Kafka community and take advantage of the many other open source projects in the ecosystem to help automate many of the tasks outlined in this chapter.

Now that we are confident in the tools needed to administer and manage our cluster, it is still impossible without proper monitoring in place.

[Chapter 13](#) will discuss ways to monitor broker and cluster health and operations so you can be sure Kafka is working well (and know when it isn't). We will also offer best practices for monitoring your clients, including both producers and consumers.